ONYX LAB · WHITEPAPER SERIES

ONYXLABWHITEPAPER

BANDWORK

Rapid Improvement and the Bindery

Version 1.5 · October 2026 · Onyx Lab · replaces the 27 Oct model-coverage edition

A measurement paper on converting assembled model output into verified artefacts. 110 builds have been judged by a real Windows compiler: 34 compiled first time (31%), and 44 produced a compiling artefact after the automatic repair ladder (40%). The strongest frontier model now holds 42% native. Cost to date $15.16, or $0.34 per working artefact.

Abstract

Bandwork is the lane that turns one request into one source file: a planner draws a skeleton, each slot is filled by a separately worded request, and the pieces are assembled locally. It exists because a frontier model asked for a whole program often refuses or derails, while the same model asked for one function at a time answers. What changed in this release is not the lane: it is the layer wrapped around it - The Bindery - which assembles deterministically, repairs one function at a time with a different model, and verifies with a real compiler. This paper states the mechanism, the measured counts with their denominators, and the limits.

01

Scope and Question

Two weeks ago the pipeline could assemble a program from eight models and almost none of them compiled, and the diagnosis was that the models were inventing code. This paper tests that diagnosis. The question is narrow: how much of the failure was the model, and how much was the machinery between the model and the compiler?

The answer in the data is that more than half was the machinery - assembly order, missing standard headers, shared globals that nobody declared, and an instrument that was misreading the compiler's own diagnostics.

02

What Changed

Assembly order. A slot is filled blind to its siblings, so a function may legitimately call a helper that a later marker defines. Nothing declares it first and C++ refuses use before declaration. The assembler now hoists a forward declaration for any name used before it is defined - deterministic, no model call, no possibility of refusal.

Missing vocabulary. Frontier models reach for std::thread, std::filesystem and std::put_time; the skeleton rarely carried those headers, because the one-function wire means no slot owns an #include line. The assembler now supplies every standard header the finished file uses.

Shared state. The planner contract now requires shared globals to be declared at file scope in the skeleton, and the assembler drops duplicates a repairer restates.

The lying instrument. Compiler diagnostics arrive inside typographic quotes. The app was decoding that output through the Windows locale codepage, so the names arrived as mojibake and the gate written to catch fabricated identifiers could never match one - broken builds were being stamped as passes. Fixing the decode turned the failing coherence test green on its own.

03

The Bindery

The lane writes pages; the Bindery makes them into a book. It is the engineering system that converts probabilistic output into a verified artefact, and it has three benches.

The binders - assembly, deterministic, free. Forward-declaration hoisting, standard-header injection, file-scope globals carried from the blueprint, duplicate-global removal. None of it asks a model anything, so none of it can refuse, stall or drift. This bench produced the largest single jump in the counts and cost nothing.

The editor's desk - repair, one function at a time. Frontier models will not repair their own work: asked to fix their output, Opus and Sonnet return nothing and Grok refuses, consistently. The failing function is therefore handed to a cooperating model with the compiler's complaint and the names it may use. One function at a time is not tidiness: handing a fixer the whole composite trips provider safety filters, measured in both directions on the same day.

The proof room - verification, evidence rather than assertion. A real Windows cross-compiler judges every build; each verdict records the checker that produced it; a failing build is reported as a draft instead of being stamped; and the ledger keeps planner, fixer and checker identity per row. A lie detector compiles an artefact to a static executable, runs it, injects synthetic keystrokes and checks whether the program recorded them - which is how 'compiles' was separated from 'works'.

04

Method and Limits

Builds come from frozen asks. Each row records the ask, the plan size, the slots filled, the checker and the repair outcome. A pass counts only when the row carries parse_tool=wsl-mingw-g++ and an independent recompile of the stored source also passes. Identical prompts flip outcome roughly one third of the time, so a single roll is a data point and never a rate; cells are reported with their denominators wherever n is small.

Known limits: most cells are n=1 to n=4; functional correctness is measured on one artefact family only; and unattended operation has never succeeded on this machine, so runs are still driven one roll at a time.

05

Results

Across all asks, 34 of 110 judged builds compiled first time (31%) and 44 were working after repair (40%).

model	cells	native	working	blocking class
claude-sonnet-5	9	56%	56%	undeclared names (2)
claude-opus-5.5	12	17%	25%	undeclared names (10); slot-fill failure
grok-4.7	10	20%	20%	undeclared names (3)
claude-sonnet-5.5	8	0%	12%	undeclared names (6); type errors
claude-opus-5	4	0%	0%	undeclared names (4)
claude-opus-4.8	2	0%	0%	type / syntax mismatch

Claude Sonnet 5 is the first frontier model to hold above half of its cells natively, and its strongest row is a nine-function build recompiled independently from stored source. Every model below that line has its blocking class named; undeclared identifiers dominate across the whole tier, which is why the planner contract now requires every declared helper to be implemented by a slot or defined inline.

06

Refresh Catalog and Adding Models

A provider's published model list drifts independently of what actually routes: ids disappear from /models while still answering, and ids appear that no shipped catalogue carries. The Models pane therefore has a Refresh control that fetches the live list, diffs it against the shipped catalogue, and reports new and vanished ids without changing anything. Reported new ids can be added with one click.

Added ids live in Lite's own overlay and never edit the shipped catalogue. Additions merge rather than replace, survive a restart, and can be cleared again. Loopholes found while hardening this and since closed: control characters inside an id (a newline would corrupt a log line and travel into a request) are stripped; a second instance holding a stale overlay can no longer clobber another's addition, because the merge reads disk first; oversized lists are bounded; and model ids reach the DOM as text, so a hostile id cannot inject markup.

07

Corrections

– Claimed “transport blackouts” four times: the carrier was answering. The real causes were a missing client timeout (SDK default 600s with two retries) and a wedged launcher.

– Reported “431 tests passed” on 27 October: false on re-run; the suite was 430 plus one failing.

– Attributed the whole failure class to invented identifiers: half was assembly order, much of the rest missing headers and shared globals.

– Stated that a per-request timeout of 150s bounds the planner: it does not, because the SDK ignores it when the client carries its own transport. A daemon-thread wall clock replaced it.

– Assumed compiling meant working: the lie detector compiled an artefact, ran it, and found it captured zero keystrokes with its collector switched off.

08

What Is Not Proven

– The frontier aggregate remains low even though its strongest member clears half; the denominators in the table are the honest reading.

– Most cells are single rolls. Repeated rolls are what turn a possibility into a rate.

– Functional behaviour is measured on one artefact family only.

– Unattended operation: every long-lived supervisor is reaped on this machine.

A

Raw Aggregate

ask id	judged	working	rate
1514bde5	35	14	40%
c8336309	24	13	54%
721ac6d4	21	9	43%
77630752	9	1	11%
20b28147	8	2	25%
281130ef	2	0	0%
e49ecafc	1	1	100%
f21a67cc	1	1	100%
c5ba1c34	1	1	100%
1c17b531	1	0	0%
512b19e9	1	0	0%
a55c7c90	1	0	0%

Ledger: ~/.onyx/onyx-lite-1.3/bandwork-matrix.jsonl · rows 193, completed 117, judged 110. Reproduce with: python bench_models.py --report and python bench_models.py --reparse.